SYNTROPY

Don't manage time. Execute it.

HACKATHON MVP IMPLEMENTATION PLAN

Created for: VibeCode 2.0 Hackathon
Platform: Next.js + Google Gemini 1.5 Pro + GCP
Generated: 2026-06-30

[Award] Syntropy Phase 3 Execution Plan Report

This report defines the implementation plan for the **Execution Plan** view of the Syntropy platform. This view shifts the app from a simple timeline dashboard to a transparent, interactive control center for the multi-agent AI system.

1. Feature Specifications & Data Model

The Execution Plan consists of three primary interactive features:

1. **Agent Status Monitor:** Displays active statuses, heartbeats, and current operations of the multi-agent layers (Router, Scheduling, Execution, Negotiation, and Memory).
2. **Task Dependency Graph:** A visual flow representing how tasks connect and how postponing or rescheduling one task impacts downstream commitments.
3. **AI Audit Log Terminal:** A live-scrolling CLI-style feed of agent execution records.

Data Expansion in `src/lib/mockData.js`

To drive this view, we will add the following data structures:

- `mockAgentStatus`: Trackers detailing what each specialized Gemini agent is currently processing.
- `mockDependencyGraph`: Nodes and dependency linkages (e.g., "Boilerplate Setup" must precede "CS-401 Submission").
- `mockAgentLogs`: Historic system logs.

2. Layout & Routing Architecture

We will create a new route: `/dashboard/execution-plan`.

```
graph TD
    Sidebar[Sidebar.js] -->|Click Execution Plan| Route[src/app/dashboard/execution-plan/page.js]
    Route --> Header[Header.js]
    Route --> Layout[Layout Grid]
    Layout --> StatusPanel[AgentStatusPanel.js]
    Layout --> GraphPanel[DependencyGraphPanel.js]
    Layout --> TerminalPanel[AgentAuditLogTerminal.js]
```

Component Breakdown:

- **`page.js`:** The entry point that secures the route with the Auth guard and mounts the components.
- **`AgentStatusPanel.js`:** Visual grids showing pulsing status indicators for each agent.
- **`DependencyGraphPanel.js`:** A visual dependency tree.
- **`AgentAuditLogTerminal.js`:** A terminal console that streams logs sequentially.

3. Step-by-Step Implementation Roadmap

Phase A: Data & Page Setup

1. Update `src/lib/mockData.js` with the status, graph, and logs datasets.
2. Create directory `src/app/dashboard/execution-plan` and create its `page.js`.
3. Update `Sidebar.js` to change the "Execution Plan" menu item path from `#` to `/dashboard/execution-plan`.

Phase B: Build Sub-Components

1. Create `AgentStatusPanel.js` displaying agent metrics.
2. Create `DependencyGraphPanel.js` showing dependency paths.
3. Create `AgentAuditLogTerminal.js` displaying streaming terminal logs.

Phase C: Assemble and Build Verify

1. Wire the sub-components into the main `/dashboard/execution-plan/page.js`.
2. Run build verification (`npm run build`) to ensure compile success.